

Fixed Income - Credit Rating Research

LINVEST21 Proprietary Credit Rating Framework

Bloomberg Global Aggregate Data-Driven Methodology v1.0

Executive Summary

This specification outlines a quantitative credit rating framework that leverages Bloomberg Global Aggregate index data to create proprietary credit ratings. The methodology combines traditional agency ratings (35% weight) with quantitative market signals (40% weight), sector risk assessments (25% weight), and proprietary alpha factors (15% weight) to generate LINVEST21 credit ratings.

Target Universe: US Aggregate securities (Phase 1) → Global IG expansion (Phase 2+) **Data Source:** Bloomberg Global Aggregate Index dataset (31,125 securities, 78 columns) **Output:** Proprietary LIN-AAA through LIN-CCC rating scale

Step 1: Data Preprocessing & Universe Filtering

Objective

Filter the Bloomberg Global Aggregate dataset to create a clean, investable universe for rating assignment.

Required Bloomberg Fields

```
1 Primary Fields:
2 - Currency          (String) - Security currency denomination
3 - QualityB          (String) - Bloomberg beginning quality rating
4 - OutstandE         (Float)  - Outstanding amount end of period
5 - Maturity          (Float)  - Years to maturity
6 - MrktValue         (Float)  - Current market value
```

Filter Logic

sql

```
1 SELECT * FROM bloomberg_global_agg
2 WHERE Currency = 'USD'
3    AND QualityB NOT IN ('NR', 'D', '')
4    AND OutstandE >= 300000000
5    AND Maturity >= 1.0
```

```
6 AND MktValue IS NOT NULL;
```

Expected Output

- **Input:** 31,125 total securities
 - **Filtered Output:** ~8,500 eligible USD investment-grade securities
 - **Exclusions:** Unrated bonds, defaulted securities, sub-\$300M issues, <1yr maturity
-

Step 2: Quantitative Risk Score Matrix (40% Total Weight)

Component 2A: Duration Risk Score (15% Weight)

Bloomberg Field: ISMA_MDur (ISMA Modified Duration)

Calculation Logic:

python

```
1 duration_risk_score = min(ISMA_MDur / 10.0, 1.0) * 100
2
3 # Risk Interpretation:
4 # 0-2 years: Low risk (20-40 points)
5 # 2-5 years: Moderate risk (40-60 points)
6 # 5-10 years: High risk (60-100 points)
7 # 10+ years: Maximum risk (100 points)
```

Rationale: Longer duration securities exhibit higher interest rate sensitivity, increasing price volatility and investment risk.

Component 2B: Credit Spread Risk Score (15% Weight)

Bloomberg Field: OAS_bp (Option-Adjusted Spread in basis points)

Calculation Logic:

python

```
1 spread_risk_score = min(OAS_bp / 500.0, 1.0) * 100
2
3 # Risk Interpretation:
4 # 0-50bp: Minimal spread risk (10-20 points)
5 # 50-150bp: Moderate spread risk (20-40 points)
6 # 150-300bp: Elevated spread risk (40-70 points)
7 # 300-500bp: High spread risk (70-100 points)
8 # 500bp+: Maximum spread risk (100 points)
```

Rationale: Higher spreads indicate market perception of increased credit risk and potential volatility.

Component 2C: Liquidity Risk Score (10% Weight)

Bloomberg Field: `OutstandE` (Outstanding Amount End)

Calculation Logic:

python

```
1 import math
2 liquidity_risk_score = 100 - min(math.log10(OutstandE / 1000000) * 20, 80)
3
4 # Risk Interpretation:
5 # $10B+: Excellent liquidity (20 points)
6 # $1-10B: Good liquidity (40 points)
7 # $500M-1B: Moderate liquidity (60 points)
8 # $300-500M: Lower liquidity (80 points)
```

Rationale: Larger outstanding amounts typically correlate with improved secondary market liquidity and lower transaction costs.

Quantitative Component Aggregation

python

```
1 quantitative_score = (duration_risk_score * 0.15) + (spread_risk_score * 0.15) +
  (liquidity_risk_score * 0.10)
```

Step 3: Sector Risk Matrix (25% Total Weight)

Required Bloomberg Fields

1	- IssrClsL1	(String)	- Issuer Class Level 1
2	- IssrClsL2	(String)	- Issuer Class Level 2
3	- Sector	(String)	- Bloomberg sector classification
4	- Country	(String)	- Country of risk

Sector Risk Multiplier Matrix

```
1 Sector CategoryIssrClsL1 CodeRisk MultiplierBase Score
ImpactTreasury/SovereignGovernment0.5025 pointsGovernment-RelatedGovt-Related0.6532.5
pointsAgency/GSEAgency0.7035 pointsCorporate-UtilityCorporate-Utility0.8542.5
pointsCorporate-IndustrialCorporate-Industrial1.0050 pointsCorporate-FinancialCorporate-
Financial1.1557.5 pointsSecuritizedSecuritized0.9045 points
```

Calculation Logic

python

```
1 def calculate_sector_score(issuer_class_l1):
2     base_score = 50 # Neutral baseline
3     sector_risk_multipliers = {
4         'Government': 0.50,
5         'Government-Related': 0.65,
6         'Agency': 0.70,
```

```

7         'Corporate-Utility': 0.85,
8         'Corporate-Industrial': 1.00,
9         'Corporate-Financial': 1.15,
10        'Securitized': 0.90
11    }
12
13    multiplier = sector_risk_multipliers.get(issuer_class_l1, 1.00)
14    return base_score * multiplier

```

Step 4: Agency Rating Synthesis (35% Total Weight)

Required Bloomberg Fields

```

1 - QualityB          (String) - Bloomberg composite rating beginning
2 - QualityE          (String) - Bloomberg composite rating ending

```

Agency Rating Translation Matrix

```

1 Bloomberg RatingNumeric ScoreLINVEST21 EquivalentAaa/AAA95LIN-AAAAa1/AA+90LIN-AA+Aa2/AA87LIN-
  AAAa3/AA-84LIN-AA-A1/A+81LIN-A+A2/A78LIN-AA3/A-75LIN-A-Baa1/BBB+72LIN-BBB+Baa2/BBB68LIN-
  BBBBaa3/BBB-65LIN-BBB-Ba1/BB+55LIN-BB+Ba2/BB50LIN-BBBa3/BB-45LIN-BB-B1/B+35LIN-B+B2/B30LIN-
  BB3/B-25LIN-B-Caa1/CCC+15LIN-CCC+Caa2/CCC10LIN-CCCCaa3/CCC-5LIN-CCC-

```

Implementation Logic

python

```

1 def agency_rating_score(quality_rating):
2     rating_scores = {
3         'Aaa': 95, 'AAA': 95,
4         'Aa1': 90, 'AA+': 90,
5         'Aa2': 87, 'AA': 87,
6         'Aa3': 84, 'AA-': 84,
7         'A1': 81, 'A+': 81,
8         'A2': 78, 'A': 78,
9         'A3': 75, 'A-': 75,
10        'Baa1': 72, 'BBB+': 72,
11        'Baa2': 68, 'BBB': 68,
12        'Baa3': 65, 'BBB-': 65,
13        'Ba1': 55, 'BB+': 55,
14        'Ba2': 50, 'BB': 50,
15        'Ba3': 45, 'BB-': 45,
16        'B1': 35, 'B+': 35,
17        'B2': 30, 'B': 30,
18        'B3': 25, 'B-': 25,
19        'Caa1': 15, 'CCC+': 15,
20        'Caa2': 10, 'CCC': 10,
21        'Caa3': 5, 'CCC-': 5
22    }
23
24    return rating_scores.get(quality_rating, 50) # Default to neutral if not found

```

Step 5: Proprietary Alpha Factors (15% Total Weight)

Component 5A: Price Momentum Score (5% Weight)

Bloomberg Fields: RetTotal, RetPrice

Calculation Logic:

python

```
1 momentum_score = 50 + (RetTotal * 10)
2
3 # Momentum bands:
4 # Positive momentum (+5% returns): 100 points
5 # Neutral (0% returns): 50 points
6 # Negative momentum (-5% returns): 0 points
7 momentum_score = max(0, min(100, momentum_score))
```

Component 5B: Market Value Stability (5% Weight)

Bloomberg Fields: MrktValue, MrkValBeg

Calculation Logic:

python

```
1 mv_change_pct = abs((MrktValue - MrkValBeg) / MrkValBeg * 100) if MrkValBeg != 0 else 0
2 stability_score = 100 - min(mv_change_pct, 100)
3
4 # Stability interpretation:
5 # 0-2% change: High stability (96-100 points)
6 # 2-5% change: Moderate stability (90-96 points)
7 # 5-10% change: Lower stability (80-90 points)
8 # 10%+ change: Volatile (0-80 points)
```

Component 5C: Currency Risk Factor (5% Weight)

Bloomberg Fields: RetCurncy, ProductCurrency

Calculation Logic:

python

```
1 currency_risk_score = 50 - abs(RetCurncy * 20)
2 currency_risk_score = max(0, min(100, currency_risk_score))
3
4 # For USD base currency securities, RetCurncy should be minimal
5 # Higher absolute currency returns indicate FX volatility risk
```

Alpha Component Aggregation

python

```
1 alpha_score = (momentum_score * 0.05) + (stability_score * 0.05) + (currency_risk_score * 0.05)
```

Step 6: Master Score Aggregation & Rating Assignment

Master Formula

python

```
1 final_linvest21_score = (  
2     (quantitative_score * 0.40) +  
3     (sector_score * 0.25) +  
4     (agency_score * 0.35) +  
5     (alpha_score * 0.15)  
6 )  
7  
8 # Normalize to 0-100 scale  
9 final_score = max(0, min(100, final_linvest21_score))
```

LINVEST21 Rating Assignment Scale

```
1 Score RangeLINVEST21 RatingCredit QualityInvestment Category90-100LIN-AAASuperiorPrime  
Investment Grade85-89LIN-AA+ExcellentHigh Investment Grade80-84LIN-AAExcellentHigh Investment  
Grade75-79LIN-AA-StrongUpper Investment Grade70-74LIN-A+StrongUpper Investment Grade65-69LIN-  
AAdequateLower Investment Grade60-64LIN-A-AdequateLower Investment Grade55-59LIN-  
BBB+AdequateLower Investment Grade50-54LIN-BBBMinimum IGLowest Investment Grade45-49LIN-BBB-  
SpeculativeHigh Yield Boundary40-44LIN-BB+SpeculativeHigh Yield35-39LIN-BBSpeculativeHigh  
Yield30-34LIN-BB-Highly SpeculativeHigh Yield25-29LIN-B+Highly SpeculativeHigh Yield20-24LIN-  
BSubstantial RiskHigh Yield15-19LIN-B-Substantial RiskHigh Yield10-14LIN-CCC+Extremely  
SpeculativeDistressed5-9LIN-CCCExtremely SpeculativeDistressed0-4LIN-CCC-Near  
DefaultDistressed
```

Step 7: Validation & Quality Control Framework

Validation Checkpoints

7.1 Bloomberg Baseline Validation

python

```
1 def validate_bloomberg_consistency(lin_score, bloomberg_quality):  
2     bloomberg_numeric = agency_rating_score(bloomberg_quality)  
3     score_difference = abs(lin_score - bloomberg_numeric)  
4  
5     if score_difference <= 15:  
6         return "VALIDATED"  
7     elif score_difference <= 25:  
8         return "REVIEW_REQUIRED"  
9     else:  
10        return "MANUAL_OVERRIDE_NEEDED"
```

7.2 Spread Consistency Check

python

```
1 def validate_spread_correlation(lin_score, oas_spread):  
2     # Higher scores should correlate with lower spreads
```

```

3     expected_spread = 500 - (lin_score * 4) # Rough linear relationship
4     spread_deviation = abs(oas_spread - expected_spread)
5
6     return "PASS" if spread_deviation < 200 else "FLAG_FOR_REVIEW"

```

7.3 Peer Group Analysis

python

```

1 def peer_group_validation(issuer, sector, lin_score):
2     # Find securities from same issuer or similar sector
3     peer_scores = get_peer_scores(issuer, sector)
4
5     if len(peer_scores) > 0:
6         peer_std = statistics.stdev(peer_scores)
7         peer_mean = statistics.mean(peer_scores)
8
9         z_score = abs(lin_score - peer_mean) / peer_std if peer_std > 0 else 0
10
11     return "OUTLIER" if z_score > 2.0 else "WITHIN_RANGE"

```

Quality Control Fields for Validation

```

1 Validation Fields:
2 - YldWorstE      (Float)   - Yield to worst end
3 - OAS_bp         (Float)   - Option-adjusted spread
4 - DurAdjMod      (Float)   - Duration adjusted modified
5 - ConvAdj        (Float)   - Convexity adjustment
6 - Issuer         (String)  - Issuer name for peer analysis

```

Technical Implementation Specification

Database Schema

sql

```

1 CREATE TABLE linvest21_credit_ratings (
2     rating_id SERIAL PRIMARY KEY,
3     cusip VARCHAR(9) NOT NULL,
4     isin VARCHAR(12),
5     calculation_date DATE NOT NULL,
6
7     -- Security Identifiers
8     issuer VARCHAR(100),
9     ticker VARCHAR(20),
10    currency VARCHAR(3),
11
12    -- Component Scores
13    quantitative_score DECIMAL(5,2),
14    duration_component DECIMAL(5,2),
15    spread_component DECIMAL(5,2),
16    liquidity_component DECIMAL(5,2),
17
18    sector_score DECIMAL(5,2),
19    agency_score DECIMAL(5,2),
20    alpha_score DECIMAL(5,2),

```

```

21
22 -- Final Rating
23 final_score DECIMAL(5,2),
24 linvest21_rating VARCHAR(10),
25
26 -- Validation Flags
27 validation_status VARCHAR(20),
28 bloomberg_deviation DECIMAL(5,2),
29 peer_z_score DECIMAL(5,2),
30
31 -- Audit Trail
32 created_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
33 last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
34 );
35
36 CREATE INDEX idx_cusip_date ON linvest21_credit_ratings(cusip, calculation_date);
37 CREATE INDEX idx_rating_date ON linvest21_credit_ratings(linvest21_rating,
    calculation_date);

```

Core Calculation Engine (Python)

python

```

1 class LINVEST21RatingEngine:
2
3     def __init__(self):
4         self.sector_risk_multipliers = {
5             'Government': 0.50,
6             'Government-Related': 0.65,
7             'Agency': 0.70,
8             'Corporate-Utility': 0.85,
9             'Corporate-Industrial': 1.00,
10            'Corporate-Financial': 1.15,
11            'Securitized': 0.90
12        }
13
14    def calculate_rating(self, bond_data):
15        """
16        Calculate LINVEST21 rating for a single bond
17
18        Args:
19            bond_data (dict): Bloomberg bond data record
20
21        Returns:
22            dict: Rating components and final score
23        """
24
25        # Step 2: Quantitative Score (40%)
26        duration_score = min(bond_data['ISMA_MDur'] / 10.0, 1.0) * 100
27        spread_score = min(bond_data['OAS_bp'] / 500.0, 1.0) * 100
28        liquidity_score = 100 - min(
29            math.log10(bond_data['OutstandE'] / 1000000) * 20, 80
30        )
31
32        quantitative_score = (
33            duration_score * 0.15 +
34            spread_score * 0.15 +
35            liquidity_score * 0.10

```



```

36     )
37
38     # Step 3: Sector Score (25%)
39     sector_multiplier = self.sector_risk_multipliers.get(
40         bond_data['IssrClsL1'], 1.00
41     )
42     sector_score = 50 * sector_multiplier * 0.25
43
44     # Step 4: Agency Score (35%)
45     agency_score = self.agency_rating_score(bond_data['QualityB']) * 0.35
46
47     # Step 5: Alpha Score (15%)
48     momentum_score = max(0, min(100, 50 + (bond_data['RetTotal'] * 10)))
49
50     mv_change = abs((bond_data['MrktValue'] - bond_data['MrkValBeg']) /
51                     bond_data['MrkValBeg'] * 100) if bond_data['MrkValBeg'] != 0 else 0
52     stability_score = 100 - min(mv_change, 100)
53
54     currency_score = max(0, min(100, 50 - abs(bond_data['RetCurncy'] * 20)))
55
56     alpha_score = (momentum_score + stability_score + currency_score) * 0.05
57
58     # Final Aggregation
59     final_score = quantitative_score + sector_score + agency_score + alpha_score
60     final_score = max(0, min(100, final_score))
61
62     return {
63         'final_score': final_score,
64         'linvest21_rating': self.score_to_rating(final_score),
65         'quantitative_score': quantitative_score,
66         'sector_score': sector_score,
67         'agency_score': agency_score,
68         'alpha_score': alpha_score,
69         'components': {
70             'duration': duration_score,
71             'spread': spread_score,
72             'liquidity': liquidity_score,
73             'momentum': momentum_score,
74             'stability': stability_score,
75             'currency': currency_score
76         }
77     }
78
79     def score_to_rating(self, score):
80         """Convert numeric score to LINVEST21 rating"""
81         if score >= 95: return 'LIN-AAA'
82         elif score >= 90: return 'LIN-AA+'
83         elif score >= 87: return 'LIN-AA'
84         elif score >= 84: return 'LIN-AA-'
85         elif score >= 81: return 'LIN-A+'
86         elif score >= 78: return 'LIN-A'
87         elif score >= 75: return 'LIN-A-'
88         elif score >= 72: return 'LIN-BBB+'
89         elif score >= 68: return 'LIN-BBB'
90         elif score >= 65: return 'LIN-BBB-'
91         elif score >= 55: return 'LIN-BB+'
92         elif score >= 50: return 'LIN-BB'
93         elif score >= 45: return 'LIN-BB-'

```

```
94     elif score >= 35: return 'LIN-B+'
95     elif score >= 30: return 'LIN-B'
96     elif score >= 25: return 'LIN-B-'
97     elif score >= 15: return 'LIN-CCC+'
98     elif score >= 10: return 'LIN-CCC'
99     else: return 'LIN-CCC-'
```

Data Processing Pipeline

Daily ETL Process

python

```
1 def daily_rating_process():
2     """Daily credit rating calculation workflow"""
3
4     # 1. Extract Bloomberg Data
5     bloomberg_data = extract_bloomberg_global_agg()
6
7     # 2. Apply Universe Filters
8     filtered_data = bloomberg_data[
9         (bloomberg_data['Currency'] == 'USD') &
10        (~bloomberg_data['QualityB'].isin(['NR', 'D', ''])) &
11        (bloomberg_data['OutstandE'] >= 3000000000) &
12        (bloomberg_data['Maturity'] >= 1.0)
13    ]
14
15    # 3. Calculate Ratings
16    rating_engine = LINVEST21RatingEngine()
17    results = []
18
19    for _, bond in filtered_data.iterrows():
20        try:
21            rating_result = rating_engine.calculate_rating(bond.to_dict())
22            rating_result['cusip'] = bond['Cusip']
23            rating_result['calculation_date'] = datetime.now().date()
24            results.append(rating_result)
25        except Exception as e:
26            log_error(f"Rating calculation failed for {bond['Cusip']}: {e}")
27
28    # 4. Validation & Quality Control
29    validated_results = []
30    for result in results:
31        validation_status = validate_rating(result, bond_data)
32        result['validation_status'] = validation_status
33        validated_results.append(result)
34
35    # 5. Store Results
36    store_ratings_to_database(validated_results)
37
38    # 6. Generate Alerts for Outliers
39    generate_outlier_alerts(validated_results)
40
41    return validated_results
```

Performance Monitoring Requirements

Daily Monitoring Metrics

python

```
1 monitoring_metrics = {
2     'coverage_rate': 'COUNT(rated_securities) / COUNT(eligible_securities)',
3     'agency_correlation': 'CORR(lin_score, bloomberg_numeric_score)',
4     'spread_correlation': 'CORR(lin_score, OAS_bp) * -1', # Should be negative
5     'outlier_rate': 'COUNT(outliers) / COUNT(total_ratings)',
6     'processing_time': 'MAX(calculation_timestamp) - MIN(start_timestamp)'
7 }
```

Weekly Validation Reports

- Rating distribution analysis vs Bloomberg baseline
 - Sector concentration and bias detection
 - Performance attribution by component
 - False positive/negative rate analysis
-

API Integration Specification

Real-Time Rating Lookup

python

```
1 GET /api/v1/rating/{cusip}
2 Response:
3 {
4     "cusip": "12345678",
5     "linvest21_rating": "LIN-AA-",
6     "final_score": 84.5,
7     "components": {
8         "quantitative": 33.6,
9         "sector": 21.25,
10        "agency": 29.4,
11        "alpha": 12.8
12    },
13    "validation_status": "VALIDATED",
14    "calculation_date": "2025-08-29",
15    "last_updated": "2025-08-29T16:30:00Z"
16 }
```

Batch Rating Processing

python

```
1 POST /api/v1/ratings/batch
2 Request Body:
3 {
4     "cusips": ["12345678", "87654321", ...],
5     "include_components": true,
6     "validation_level": "full"
7 }
```

Implementation Timeline & Resource Requirements

Phase 1: Core Rating Engine (6 weeks)

- Bloomberg data ingestion pipeline
- Rating calculation engine implementation
- Basic validation framework
- Unit testing and QA

Resources: 2 Senior Developers, 1 Data Engineer

Phase 2: Analytics & Monitoring (4 weeks)

- Real-time dashboard development
- Performance tracking system
- Outlier detection algorithms
- API endpoint creation

Resources: 1 Frontend Developer, 1 Backend Developer, 1 DevOps Engineer

Phase 3: Global Expansion (8 weeks)

- Multi-currency support (EUR, JPY, GBP)
- Regional sector risk calibration
- Enhanced validation logic
- Production deployment

Resources: 2 Senior Developers, 1 Risk Analyst

Risk Management & Governance

Model Risk Controls

- **Backtesting:** Monthly performance validation against historical default rates
- **Stress Testing:** Quarterly scenario analysis under extreme market conditions
- **Peer Review:** Semi-annual methodology review by independent risk committee
- **Regulatory Alignment:** Annual assessment vs agency rating methodologies

Operational Controls

- **Data Quality:** Real-time Bloomberg feed validation
- **Calculation Accuracy:** Daily reconciliation vs manual calculations

- **System Redundancy:** Backup rating engines for business continuity
- **Audit Trail:** Complete calculation history for regulatory purposes

Appendix: Bloomberg Field Mapping Reference

Complete Field Utilization Matrix

1	Framework ComponentBloomberg FieldData TypeUsageFallbackUniverse FilterCurrencyStringPrimary filterN/AQualityBStringExclude unratedUse QualityEOutstandEFloatLiquidity thresholdUse OutstandBMaturityFloatMaturity filterCalculate from MaturDateQuantitative (40%)ISMA_MDurFloatDuration riskUse DurAdjModOAS_bpFloatSpread riskUse YldWorstE differentialOutstandEFloatLiquidity scoringUse MrktValue proxySector (25%)IssrClsL1StringPrimary classificationUse IssrClassIssrClsL2StringSecondary classificationUse SectorCountryStringGeographic riskUse currency mappingAgency (35%)QualityBStringRating synthesisUse QualityEAlpha (15%)RetTotalFloatMomentum scoringCalculate from componentsMrktValueFloatStability analysisUse PriceE × OutstandEMrkValBegFloatChange calculationUse PriceB × OutstandBRetCurncyFloatFX impactSet to 0 for USD base
---	---

Data Quality Thresholds

- **Minimum Coverage:** 95% of eligible universe must have complete data
- **Calculation Success:** 98% of ratings must complete without errors
- **Validation Pass Rate:** 90% of ratings must pass all validation checks
- **Performance Target:** Complete daily processing within 2 hours

This specification provides the complete framework for implementing LINVEST21's proprietary credit rating system using Bloomberg Global Aggregate data. The methodology balances quantitative market signals with traditional credit assessment while maintaining institutional-grade rigor and transparency.